

ALOHA Operation Guide — ORPA (SIM)

1. Overview

This guide covers the ORPA simulation workflow using the ALOHA simulation environment. The workflow begins by setting up the simulation and recording good demonstrations, followed by generating bad or perturbed demonstrations with controlled errors.

The collected datasets can then be visualized and verified before training the base ACT policy. Afterward, the ORPA feedback modules are trained using the labeled perturbation data, and the complete system is evaluated to measure the effectiveness of feedback-based policy correction.

2. Run the Simulation Environment

Start the simulation:

```
python3 sim_env.py
```

To run the simulation for a fixed number of steps and save the trajectory:

```
python3 sim_env.py \  
  --num_steps 1000 \  
  --save_hdf5 ./datasets/teleop_good/episode_0.hdf5 \  
  --camera_names top
```

This can be used to manually inspect the simulation and record a trajectory directly to HDF5.

3. Record Good Demonstrations

Transfer Cube

Record 50 good demonstrations:

```
python3 record_sim_episodes.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted/ \  
  --num_episodes 50 \  
  --onscreen_render
```

Note: This is only for good examples.

Insertion

Record 50 good demonstrations:

```
python3 record_sim_episodes.py \  
  --task_name sim_insertion_scripted \  
  --dataset_dir ./datasets/sim_insertion_scripted/ \  
  --num_episodes 50 \  
  --onscreen_render
```

Note: This is also only for good examples.

4. Record Bad/Perturbed Demonstrations

ORPA requires bad demonstrations with known feedback labels in addition to the good demonstrations.

Discrete Translation Noise

Example — `too left`:

```
python3 record_sim_episodes_bad_discrete_translation.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_discrete/ \  
  --num_episodes 1 \  
  --onscreen_render \  
  --noise_type too_left
```

Examples of translation feedback include: `too_left`, `too_right`, `too_far`, `too_close`, `too_high`, `too_low`.

Discrete Rotation Noise

Example — `small left rotation`:

```
python3 record_sim_episodes_bad_discrete_rotation.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_discrete/ \  
  --num_episodes 50 \  
  --onscreen_render \  
  --noise_type rotate_left_small
```

Rotation feedback can include: `rotate_right_small`, `rotate_right_medium`, `rotate_right_large`, `rotate_left_small`, `rotate_left_medium`,

rotate_left_large.

Continuous Noise

Example:

```
python3 record_sim_episodes_bad_continuous.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_continuous/ \  
  --num_episodes 50 \  
  --onscreen_render \  
  --noise_type too_left
```

5. Old Evaluation Data Recording

The following scripts were previously used to generate separate evaluation datasets.

Discrete

```
python3 record_sim_episodes_bad_eval.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_discrete/ \  
  --num_episodes 10 \  
  --onscreen_render \  
  --noise_type too_left \  
  --noise_data discrete
```

Continuous

```
python3 record_sim_episodes_bad_eval.py \  
  --task_name sim_transfer_cube_scripted \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_continuous/ \  
  --num_episodes 10 \  
  --onscreen_render \  
  --noise_type too_left \  
  --noise_data continuous
```

Note: These commands are **not necessary anymore**.

6. Visualize the Dataset

Visualize a specific episode:

```
python3 visualize_episodes.py \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted/too_left/ \  
  --episode_idx 0
```

Visualize the right-arm joint-position distribution:

```
python3 visualize_episodes.py \  
  --dataset_dir  
  ./datasets/sim_transfer_cube_scripted_continuous/too_left/ \  
  --plot_right_arm_qpos_dist
```

Use visualization to verify that the recorded perturbation and feedback labels are correct before training.

7. Train the Base ACT Policy

Discrete Dataset

```
python3 imitate_episodes.py \  
  --task_name sim_transfer_cube_scripted \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_discrete \  
  --policy_class ACT \  
  --kl_weight 10 \  
  --chunk_size 100 \  
  --batch_size 8 \  
  --hidden_dim 512 \  
  --dim_feedforward 3200 \  
  --num_epochs 6000 \  
  --lr 1e-5 \  
  --seed 0 \  
  --noise_data discrete
```

Continuous Dataset

```
python3 imitate_episodes.py \  
  --task_name sim_transfer_cube_scripted \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_continuous \  
  --policy_class ACT \  
  --kl_weight 10 \  
  --chunk_size 100 \  
  --batch_size 8 \  
  --hidden_dim 512 \  
  --dim_feedforward 3200 \  
  --num_epochs 6000 \  
  --noise_data continuous
```

```
--lr 1e-5 \  
--seed 0 \  
--noise_data continuous
```

The ACT policy is trained first and used as the base policy before training the ORPA feedback modules.

8. Evaluate the Base ACT Policy

Discrete

```
python3 imitate_episodes.py \  
  --task_name sim_transfer_cube_scripted \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_discrete \  
  --policy_class ACT \  
  --kl_weight 10 \  
  --chunk_size 100 \  
  --batch_size 8 \  
  --hidden_dim 512 \  
  --dim_feedforward 3200 \  
  --num_epochs 6000 \  
  --lr 1e-5 \  
  --seed 0 \  
  --noise_data discrete \  
  --eval
```

Continuous

```
python3 imitate_episodes.py \  
  --task_name sim_transfer_cube_scripted \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_continuous \  
  --policy_class ACT \  
  --kl_weight 10 \  
  --chunk_size 100 \  
  --batch_size 8 \  
  --hidden_dim 512 \  
  --dim_feedforward 3200 \  
  --num_epochs 6000 \  
  --lr 1e-5 \  
  --seed 0 \  
  --noise_data continuous \  
  --eval
```

9. Train ORPA Feedback Modules

After training the base ACT policy, train the Feedback Encoder / Policy Updater modules.

Discrete Translation Feedback

```
python3 train_feedback_modules_transformer.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_discrete \
  --policy_class ACT \
  --kl_weight 10 \
  --chunk_size 100 \
  --batch_size 8 \
  --hidden_dim 512 \
  --dim_feedforward 3200 \
  --num_epochs 2000 \
  --lr 1e-5 \
  --fe_pu_lr 1e-5 \
  --seed 0 \
  --dataset_dir ./datasets/sim_transfer_cube_scripted_discrete \
  --feedback_vocab
good,too_left,too_right,too_far,too_close,too_high,too_low \
  --realtime_plot \
  --noise_data discrete
```

Discrete Translation + Rotation Feedback

```
python3 train_feedback_modules_transformer.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_discrete \
  --policy_class ACT \
  --kl_weight 10 \
  --chunk_size 100 \
  --batch_size 8 \
  --hidden_dim 512 \
  --dim_feedforward 3200 \
  --num_epochs 2000 \
  --lr 1e-5 \
  --fe_pu_lr 1e-5 \
  --seed 0 \
  --dataset_dir ./datasets/sim_transfer_cube_scripted_discrete \
  --feedback_vocab
good,too_left,too_right,too_far,too_close,too_high,too_low,rotate_right_
small,rotate_right_medium,rotate_right_large,rotate_left_small,rotate_le
ft_medium,rotate_left_large \
  --realtime_plot \
```

```
--noise_data discrete
```

This version additionally includes rotational feedback.

Continuous Feedback

```
python3 train_feedback_modules_transformer.py \  
  --task_name sim_transfer_cube_scripted \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_continuous \  
  --policy_class ACT \  
  --kl_weight 10 \  
  --chunk_size 100 \  
  --batch_size 8 \  
  --hidden_dim 512 \  
  --dim_feedforward 3200 \  
  --num_epochs 2000 \  
  --lr 1e-5 \  
  --fe_pu_lr 1e-5 \  
  --seed 0 \  
  --dataset_dir ./datasets/sim_transfer_cube_scripted_continuous \  
  --feedback_vocab  
good,too_left,too_right,too_far,too_close,too_high,too_low \  
  --realtime_plot \  
  --noise_data continuous
```

10. Evaluate the Feedback Modules

Basic Feedback Module Evaluation

```
python3 eval_feedback_modules.py \  
  --task_name sim_transfer_cube_scripted_discrete \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted_discrete \  
  --policy_class ACT \  
  --num_epochs 1 \  
  --seed 0 \  
  --feedback_vocab  
good,too_left,too_right,too_far,too_close,too_high,too_low
```

11. Evaluate Loaded Feedback Modules

Discrete Translation

```
python3 eval_feedback_modules_load.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted \
  --bad_dataset_dir ./datasets/sim_transfer_cube_scripted \
  --policy_class ACT \
  --num_epochs 1 \
  --seed 0 \
  --feedback_vocab
good,too_left,too_right,too_far,too_close,too_high,too_low \
  --noise_type too_left \
  --noise_data discrete
```

Rotation Feedback

```
python3 eval_feedback_modules_load.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted \
  --bad_dataset_dir ./datasets/sim_transfer_cube_scripted \
  --policy_class ACT \
  --num_epochs 1 \
  --seed 0 \
  --feedback_vocab
good,too_left,too_right,too_far,too_close,too_high,too_low,rotate_right_
small,rotate_right_medium,rotate_right_large,rotate_left_small,rotate_le
ft_medium,rotate_left_large \
  --noise_type rotate_right_small \
  --noise_data continuous
```

Continuous Translation

```
python3 eval_feedback_modules_load.py \
  --task_name sim_transfer_cube_scripted \
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted \
  --policy_class ACT \
  --num_epochs 1 \
  --seed 0 \
  --feedback_vocab
good,too_left,too_right,too_far,too_close,too_high,too_low \
  --noise_type too_left \
  --noise_data continuous
```

12. Evaluate IK Baselines

ORPA can also be compared with inverse-kinematics correction baselines.

Translation Baseline

```
python3 eval_ik_baseline_translation.py \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted \  
  --task_name sim_transfer_cube_scripted \  
  --bad_dataset_dir ./datasets/sim_transfer_cube_scripted \  
  --noise_type too_left \  
  --noise_data discrete
```

Rotation Baseline

```
python3 eval_ik_baseline_rotation.py \  
  --ckpt_dir ./checkpoints/sim_transfer_cube_scripted \  
  --task_name sim_transfer_cube_scripted \  
  --bad_dataset_dir ./datasets/sim_transfer_cube_scripted \  
  --noise_type rotate_left_large \  
  --noise_data discrete
```

13. Evaluate OLAF Baseline

Besides the IK baseline, ORPA can also be compared with OLAF. The OLAF workflow converts the ALOHA perturbation dataset into OLAF format, performs action relabeling using an LLM critic, and converts the relabeled data back into the ALOHA format for comparison.

Step 1 — Convert ALOHA Dataset to OLAF Format

Example using the `too_left` perturbation:

```
python3 lflf/scripts/convert_aloha_to_olaf.py \  
  --aloha_dataset_dir \  
datasets/sim_transfer_cube_scripted_discrete/too_left \  
  --output_dir olaf/data \  
  --task_name transfer_cube \  
  --noise_type too_left \  
  --max_episodes 5 \  
  --intervention_timestep 250 \  
  --pre_window_length 50
```

This converts the selected ALOHA trajectories into the format required by OLAF.

Step 2 — Merge OLAF Demonstrations

Merge the converted demonstrations into a single HDF5 file:

```
python3 lflf/scripts/merge_olaf_demos.py \
  --input_dir olaf/data \
```

Step 3 — Relabel Actions

OLAF uses an LLM critic to select or relabel corrective actions.

Single Trajectory

Use this command to test relabeling on one trajectory:

```
python3 lflf/scripts/relabeling_actions_single_traj.py \
  --original_dataset ./data.hdf5 \
  --reabeled_dataset ./data_reabeled.hdf5 \
  --task_name aloha_transfer_cube \
  --mode llm_critic \
  --candidate_strategy onedim \
  --num_actions 8 \
  --gpt_model gpt-3.5-turbo \
  --ep demo_0 \
  --intv_label_value -10
```

Complete Dataset

Relabel all trajectories:

```
python3 lflf/scripts/relabeling_actions.py \ --original_dataset
./data.hdf5 \ --reabeled_dataset ./data_reabeled.hdf5 \ --task_name
aloha_transfer_cube \ --mode llm_critic \ --candidate_strategy onedim \
--num_actions 8 \ --gpt_model gpt-3.5-turbo \ --intv_label_value -10
```

Step 4 — Convert Relabeled OLAF Data Back to ALOHA

After action relabeling, convert the OLAF dataset back into ALOHA format:

```
python3 lflf/scripts/convert_olaf_to_aloha.py \
  --olaf_dataset olaf/data_reabeled.hdf5 \
  --output_dir olaf/back_to_aloha \
  --max_episodes 5
```

The converted trajectories can then be used for evaluation and comparison with the ORPA correction approach.

ORPA Simulation Workflow

The ORPA simulation workflow starts by recording good demonstrations and generating controlled bad demonstrations with translation or rotation errors. The base ACT policy is

trained first, followed by the ORPA feedback modules using the labeled feedback dataset. The resulting system can then be evaluated under different perturbations and compared against IK-based correction and OLAF baselines.